

INFORME TÉCNICO · CONFIDENCIAL

Plan de Implementación de Observabilidad

Istio (Ambient) + Kiali en `tocpait-sat-dev`

Service Mesh sin modificar las aplicaciones · Entrega GitOps vía ArgoCD

Entorno	SAT dev — cluster RKE2
Cluster	<code>tocpait-sat-dev</code>
Acceso	SSH al master <code>tocpait-sat-dev-mstrs-k8s-01</code>
Identificación	pa = Panamá
Repo GitOps	tigo-devops-panama
Fecha del análisis	25 de junio de 2026

Telemetría máxima · cero cambios en las apps · Ambient data-plane

1. Contexto y problema

El cluster SAT dev no tiene service mesh ni observabilidad de tráfico: hoy no se sabe quién llama a quién, ni latencia, ni tasa de error, ni mTLS entre servicios.

El cluster **SAT dev** (`tocpait-sat-dev` , RKE2, acceso por SSH al master `tocpait-sat-dev-mstrs-k8s-01`) concentra todo su workload en un único namespace. Hoy conviven **56 deployments de negocio + el propio ArgoCD + RabbitMQ**, todos en el namespace `argocd` , expuestos por un único ingress nginx.

Lo que más importa	Situación actual
Aislamiento de workloads	56 apps + ArgoCD + RabbitMQ en el mismo ns <code>argocd</code> — anti-pattern, sin aislamiento de blast-radius
Exposición	Un único ingress nginx para todo
Visibilidad de tráfico	Inexistente — no se conoce el grafo de llamadas servicio-a-servicio
Métricas L4/L7	Sin latencia, sin tasa de error, sin RPS
Seguridad de tránsito	Sin mTLS ni identidad entre servicios
Diagramas existentes	En gran parte inferidos, no evidenciados

CRÍTICO Namespace `argocd` compartido

Tener 56 deployments de negocio junto al control-plane de ArgoCD y a RabbitMQ en un mismo namespace elimina cualquier aislamiento de blast-radius. Esto condiciona directamente la estrategia de habilitación de la telemetría (ver Fase 3) — no se puede enrollar el ns completo de una sola vez sin arriesgar al propio ArgoCD/RabbitMQ.

2. Objetivo

Subir **Istio + Kiali** en SAT dev siguiendo el patrón GitOps del repo `tigo-devops-panama` , con tres metas concretas:

1. Obtener el **máximo de telemetría sin modificar ninguna aplicación**.
2. Exponer Kiali por un **listener del propio mesh** (no por nginx).
3. Tener el **dashboard de Kiali funcionando** con grafo, health y badges de mTLS.

3. Decisiones confirmadas con el cliente

Decisión	Detalle
Modo de data-plane: Ambient	ztunnel + istio-cni, activado por <i>label de namespace</i> . No inyecta sidecar, no reinicia ni altera los pods de las apps. Entrega grafo L4 + mTLS + identidad en Kiali. L7 (HTTP/RPS/latencia) queda para fase opcional vía waypoint.
Entrega: GitOps vía ArgoCD	Versionar todo en <code>cluster-resources/</code> y dejar que el App-of-Apps existente sincronice.
Prometheus	Verificar en pre-flight; si no existe, provisionar uno dedicado a Istio.
Ubicación de Kiali	Fijado en el nodo del ArgoCD vía <code>nodeSelector</code> /affinity — ubicación determinística en dev.

4. Por qué Ambient y no Sidecar

Istio sólo genera telemetría si hay **data-plane**. La elección del modo de data-plane es la decisión de diseño más importante de este plan, porque determina si las apps se tocan o no.

Aspecto	Sidecar	Ambient ELEGIDO
Mecanismo	Inyecta un Envoy en cada pod	istio-cni + ztunnel (DaemonSet por nodo) interceptan el tráfico
Activación	Recrear/reiniciar cada pod	Sólo etiquetar el namespace
Impacto en las apps	Exige restart — viola "no alterar las aplicaciones"	Cero — sin contenedor extra, sin restart, manifiesto intacto
Telemetría inmediata	L7 completo (HTTP/RPS/latencia)	L4 + mTLS + identidad; L7 después vía waypoint (tampoco toca los pods)
Riesgo en ns <code>argocd</code>	Alto — recrear pods de ArgoCD + RabbitMQ	Bajo — no se recrean pods

ALTO Riesgo a validar: madurez de Ambient sobre el CNI del RKE2

Ambient corre sobre el CNI del cluster. RKE2 trae **Canal** por defecto, y la compatibilidad de `istio-cni` / `ztunnel` debe validarse en el pre-flight (Fase 0, paso 3). **Fallback:** si resulta incompatible, sidecar únicamente en un ns canary (cobertura parcial) — a confirmar con el cliente antes de aplicarlo.

5. Patrón del repo tigo-devops-panama (hallazgos)

- **Estructura:** Kustomize + ArgoCD **App-of-Apps** + **ApplicationSet**
(`applicationsets/roots.yaml` , generator `apps/overlays/**/api/*`).

- **Add-ons de plataforma** en `cluster-resources/`: `keycloak`, `redis`, `rabbitmq`, `nfs`, `namespace-hardening`.
- El repo **ya está "istio-ready"**: existe `cluster-resources/namespace-hardening/argocd/networkpolicy-allow-istio.yaml` que referencia `istio-system`, pero **nada de Istio está instalado**.
- **Ingress**: `ingressClassName: nginx`; TLS termina en HAProxy externo (`ssl-redirect: false`); secret `tigo-tls-secret`; host `k8s-<env>-sat.tigo.com.pa`.

6. Fases de ejecución

Fase 0 — Pre-flight (sólo lectura, en el bastion vía SSH al master)

Se ejecuta en `tocpait-sat-dev-mstrs-k8s-01` con el kubeconfig del RKE2. No muta nada.

1. ¿Existe **Prometheus/Grafana**? Define si se reaprovecha o se provisiona uno nuevo.
2. **Nodo del ArgoCD** — para el `nodeSelector` de Kiali.
3. **CNI del RKE2** (Canal) — validar compatibilidad de `istio-cni` / `ztunnel`.
4. **StorageClass / IngressClass** (confirmar `nginx`).
5. **Egress de imágenes**: `docker.io/istio/*`, `quay.io/kiali/*`, o mirror interno si airgapped.
6. **Versión de K8s/RKE2** — compatibilidad de la versión de Istio a fijar.

Si el cluster es airgapped, espejar las imágenes `istio/{pilot,install-cni,ztunnel,proxymv2}`, `quay.io/kiali/kiali` y Prometheus en el registry interno antes de la Fase 2.

Fase 1 — Estructura GitOps (sin aplicar nada aún)

Crear `cluster-resources/istio/` siguiendo el estilo de `cluster-resources/keycloak/`:

```
cluster-resources/istio/
├─ istio-base/           CRDs + base (chart istio/base, versión fija)
├─ istiod/              control-plane, profile=ambient
├─ istio-cni/           necesario para ambient
├─ ztunnel/            DaemonSet, data-plane ambient
├─ istio-ingressgateway/ gateway para exponer Kiali por el mesh
├─ prometheus/          SÓLO si la Fase 0 indica que falta
└─ kiali/              CR + nodeSelector del nodo ArgoCD + external_services.prometheus.url
```

- **AppProject** `istio` con permiso para `istio-system`, `istio-cni`, `kiali`.
- **Incluir** el nuevo directorio en `cluster-resources/kustomization.yaml`.
- **Ampliar NetworkPolicies** (la base `networkpolicy-allow-istio.yaml` ya existe) para Kiali↔Prometheus↔istiod.

Fase 2 — Bootstrap del mesh (control-plane + data-plane), sin tocar las apps

1. ArgoCD sincroniza `istio-base` → `istiod (ambient)` → `istio-cni` → `ztunnel`. Ningún namespace de app está enrolado todavía → **impacto cero** en los workloads.
2. Validar: `istioctl x precheck`, `istioctl version`, `ztunnel` `Running` en todos los nodos.
3. (Si aplica) subir el Prometheus dedicado a Istio y confirmar el scrape de los componentes del mesh.

Fase 3 — Habilitar telemetría ambient (label de namespace, sin restart)

1. Empezar por **1 namespace de bajo riesgo (canary)**; preferir un ns dedicado (p.ej. `sat-apps`) o un ns de prueba antes de tocar `argocd`.
2. `kubectl label namespace <ns> istio.io/dataplane-mode=ambient` → `ztunnel` captura el tráfico L4. Los pods **no reinician**.
3. Validar en Kiali que aparecen nodos/aristas L4 + candado mTLS para ese namespace.
4. Expandir gradualmente. mTLS permisivo primero; `PeerAuthentication STRICT` por namespace sólo después de validar.

CRÍTICO No enrolar el ns `argocd` de una sola vez

Como el ns `argocd` aloja el propio ArgoCD y RabbitMQ además de las 56 apps, etiquetarlo entero sin validación previa arriesga el control-plane de GitOps y la mensajería. Validar primero en un ns dedicado/canary y medir el impacto antes de expandir.

Fase 4 — Exponer Kiali + dashboard

1. **Listener service-mesh:** `Gateway` + `VirtualService` (`istio-ingressgateway`), host `kiali-dev-sat.tigo.com.pa` apuntando al Service de Kiali. Reusar `tigo-tls-secret`; SSL termina en HAProxy externo (`ssl-redirect: false`). Coexiste con `nginx` (no lo sustituye).
2. **Auth de Kiali:** `anonymous` / `token` en dev para acceso rápido; `openid` con el Keycloak existente en una fase posterior.
3. **Dashboard:** validar la UI de Kiali (grafo, health, mTLS badges). Opcional: importar los dashboards Grafana oficiales de Istio si hay Grafana, apuntando al mismo Prometheus.

7. Verificación end-to-end

Comprobación	Criterio de éxito
<code>istioctl x precheck</code> / <code>istioctl version</code>	Sin errores
Pods del mesh	istiod, istio-cni y ztunnel en <code>Running</code>
Namespace enrolado	<code>kubectl get ns --show-labels grep ambient</code> lista el/los ns
Dashboard Kiali	Grafo con aristas L4 + candado mTLS , sin warnings de "missing Prometheus"
Prueba de que las apps no cambiaron	<code>kubectl get pods -n <ns> : RESTARTS</code> inalterado y <code>READY</code> sigue <code>1/1</code> (no <code>2/2</code>) — sin contenedor extra
ArgoCD	Applications de Istio/Kiali <code>Synced</code> / <code>Healthy</code>
Ubicación de Kiali	Agendado en el nodo del ArgoCD

8. Riesgos y notas

Severidad	Riesgo	Mitigación
ALTO	Ambient sobre RKE2/Canal — madurez/compatibilidad del data-plane	Validar en pre-flight (Fase 0, paso 3); fallback a sidecar sólo en ns canary, previa confirmación del cliente
CRÍTICO	ns <code>argocd</code> compartido (apps + ArgoCD + RabbitMQ)	No enrolar de una vez; ns dedicado/canary primero, medir impacto, expandir gradual
MED	Egress / airgap de imágenes Istio/Kiali/Prometheus	Mirror interno antes de la Fase 2
MED	Versión de Istio vs K8s del RKE2	Fijar versión compatible (pre-flight, paso 6)

Anexo — Archivos de referencia (repo tigo-devops-panama)

- `applicationsets/roots.yaml` — modelo de Application / App-of-Apps.
- `cluster-resources/kustomization.yaml` — dónde incluir `istio/`.
- `cluster-resources/keycloak/stable/` — ejemplo de add-on bien estructurado.
- `cluster-resources/namespace-hardening/argocd/networkpolicy-allow-istio.yaml` — NetPol Istio existente, a ampliar.
- `cluster-resources/argocd-projects/argocd-projects.yaml` — dónde declarar el AppProject del mesh.

